Hyperdimensional Computing / Vector Symbolic Architectures

Slides adapted from: https://www.youtube.com/watch?v=zUCoxhExe0o

Agenda

1. Motivation
2. Basics
3. Example(s)
4. Takeaways

Motivation - Brain side

1. Brains are complex
    a. Sensory signals are variable & noisy
    b. Neurons malfunction and die
    c. Brain is flexible
2. Would be cool to model them

Clues:

- 40 billion neurons
- 240 trillion synapses
- 6000 connections per neuron on average

Motivation - Requirements

Needs a theory of computing, without it it’s nearly impossible to reverse-engineer

Requirements:

- Speed of learning
- Retention over a lifetime
- Generalisation from examples
- Reasoning by analogy
- Tolerance for variability and noise in the data.

History of computing with high dimensional vectors

- 1950s - Von Neumann: The Computer and the Brain
- 1960s - Rosenblatt: Perceptron
- 1970s and ‘80s: ANNs / Connectionism
- 1990s - Plate: Holographic Reduced Representation

Why now:

- Technology is ripe
- We need to supply the compatible theory of computing

Basics

Hyper = High dimensional

Basics

Hyper = High dimensional

Basics

Hyper = High dimensional (10.000+)

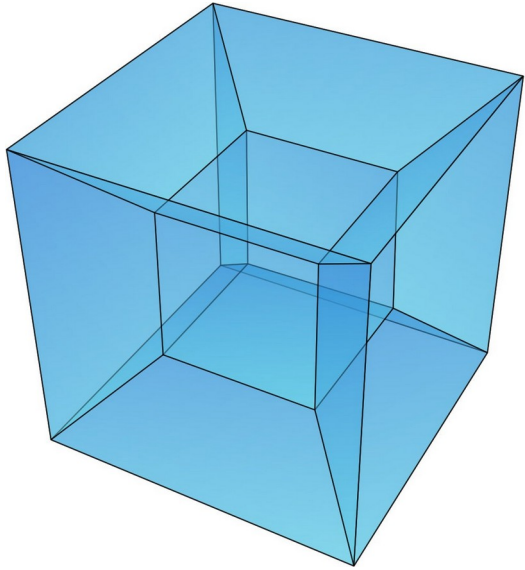
Basics

Hyper = High dimensional (10.000+)

Randomness is important

Interesting properties of Hyperdimensional Vectors

- Can be imagined as a 10.000 hypercube

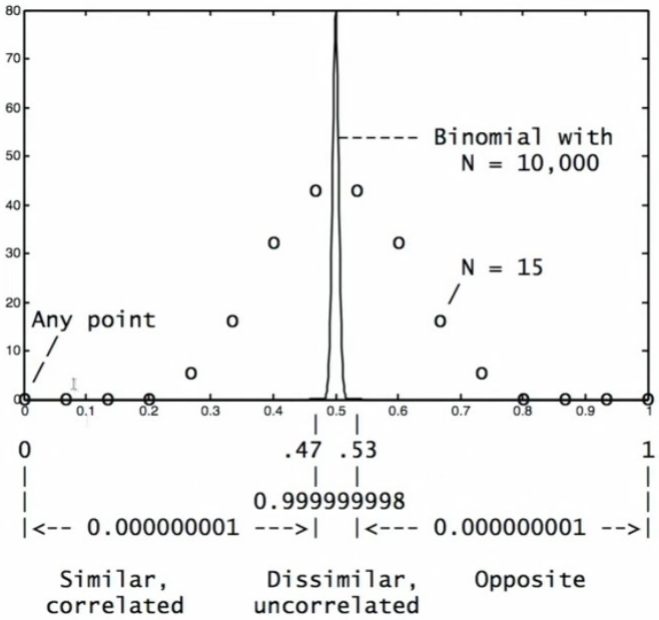


Interesting properties of Hyperdimensional Vectors

- Can be imagined as a 10.000 hypercube
- Has 10.000 mutually orthogonal vectors, but **billions** of nearly orthogonal vectors
- Retrieval works based on similarity
- “Noise” means dissimilar/orthogonal to any known vector (similarity ~0.5)

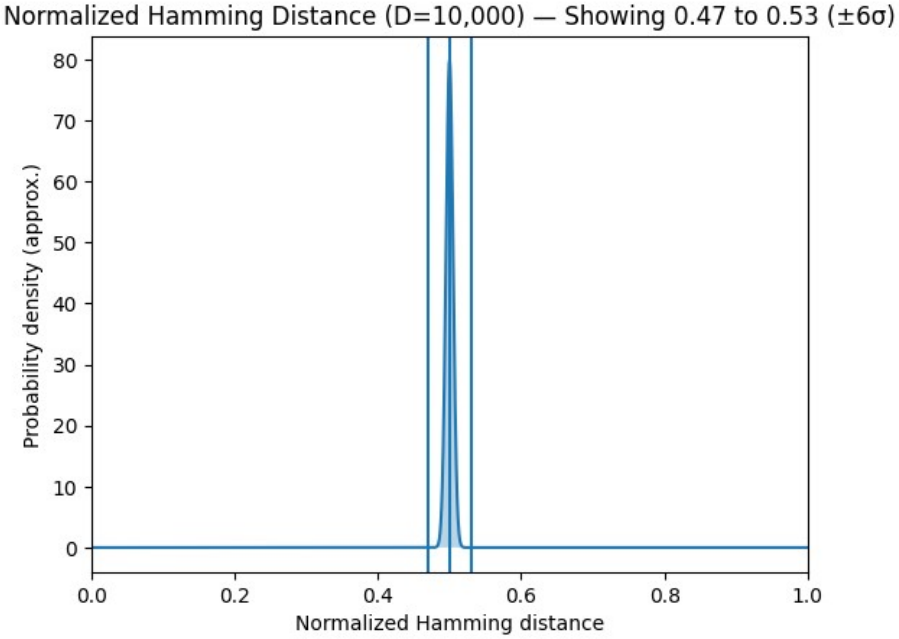
Interesting properties of Hyperdimensional Vectors

- Can be imagined as a 10.000 hypercube
- Has 10.000 mutually orthogonal vectors, but **billions** of nearly orthogonal vectors
- Retrieval works based on similarity
- “Noise” means dissimilar/orthogonal to any known vector (similarity ~0.5)
  - 99.999997% lie in 0.5 ± 0.03
  - 2 in a billion lie outside



Interesting properties of Hyperdimensional Vectors

- Can be imagined as a 10.000 hypercube
- Has 10.000 mutually orthogonal vectors, but **billions** of nearly orthogonal vectors
- Retrieval works based on similarity
- “Noise” means dissimilar/orthogonal to any known vector (similarity ~0.5)
  - 99.999997% lie in 0.5 ± 0.03
  - 2 in a billion lie outside



Arithmetic

- “ADD” → Coordinatewise majority
  - Also called ‘Boundling’
  - Produces similar vectors similar to both (superposition)
- ”MULTIPLY” → Coordinatewise XOR: M = A*B
  - Also called ‘Binding’
  - Produces dissimilar (≈orthogonal) vectors
  - Tells if the bits are different
  - XOR is it’s own inverse (A * A = 0)
  - Distributes over addition
- “PERMUTE” → rotate/shuffle vector coordinates P = rA
  - Preserves distance
  - Matrix form (not binary)
  - Used to encode sequences
- Hamming distance or cosine for similarity
- Resulting vectors have the same dimension

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Arithmetic - Key takeaways

- **Randomly** chosen vectors are *almost* **orthogonal**
- Addition produces **similar** vectors (take the majority)
- Multiplication produces **dissimilar** vectors
- Multiplication is invertible and distributes
- Permutation is also invertible and distributes
- Multiplication and permutation preserve similarity

Switching between binary to bipolar

Binary 0 ← → Bipolar 1

Binary 1 ← → Bipolar -1

XOR ← → Multiply

Majority ← → Sign

Example(s)

Paper:

- Context vector
- Rule learning
- Analogy

Lecture:

- Language example

Proc. 10th Int'l Conference on Quantum Interaction
(QI 2016, July 20-22, San Francisco)

Language Geometry using Random Indexing

Aditya Joshi[1], Johan T. Halseth[2], and Pentti Kanerva[3]

[1] Department of Mathematics, University of California–Berkeley
[2] Department of Computer Science, University of California–Berkeley
[3] Redwood Center for Theoretical Neuroscience

Abstract. Random Indexing is a simple implementation of Random Projections with a wide range of applications. It can solve a variety of problems with good accuracy without introducing much complexity. Here we demonstrate its use for identifying the language of text samples, based on a novel method of encoding letter N-grams into high-dimensional Language Vectors. Further, we show that the method is easily implemented and requires little computational power and space. As proof of the method's statistical validity, we show its success in a language-recognition task. On a difficult data set of 21,000 short sentences from 21 different languages, we achieve 97.4% accuracy, comparable to state-of-the-art methods.

Example - Motivation

People can identify languages without knowing the language

METHOD

- Compute a 10.000 dimensional profile vector for each language and for each test sentence
- Find the nearest language profile per sentence

Example - Data

- 21 EU languages
- Transcribe to latin alphabet, remove special letters
- “Train” with million bytes of text per language
- Test with 1000 sentences per language (from independent source)

Example - Computing the Profiles

Step 1) Encoding the letters

- Encode letters with 27 vectors
  - 10k dim random vectors

```
A  =  (-1 +1 -1 +1 +1 +1 -1 ... +1 +1 -1)
B  =  (+1 -1 +1 +1 +1 -1 +1 ... -1 -1 +1)
C  =  (+1 -1 +1 +1 -1 -1 +1 ... +1 -1 -1)
   ...
Z  =  (-1 -1 -1 -1 +1 +1 +1 ... -1 +1 -1)
#  =  (+1 +1 +1 +1 -1 -1 +1 ... +1 +1 -1)
```

stands for the space

Example - Computing the Profiles

Step 1) Encoding the letters

- Encode letters with 27 vectors

  - 10k dim random vectors

```
A  =  (-1 +1 -1 +1 +1 +1 -1 ... +1 +1 -1)
B  =  (+1 -1 +1 +1 +1 -1 +1 ... -1 -1 +1)
C  =  (+1 -1 +1 +1 -1 -1 +1 ... +1 -1 -1)
   ...
Z  =  (-1 -1 -1 -1 +1 +1 +1 ... -1 +1 -1)
#  =  (+1 +1 +1 +1 -1 -1 +1 ... +1 +1 -1)
```

Step 2) Encode text as trigrams

```
# stands for the space
```

- Use permutation to have a sequence (rotate by right by 1): THE= rrT * rH * E
- “The quick brown fox jumped over…”

Example - Computing the Profiles

Step 1) Encoding the letters

- Encode letters with 27 vectors
  - 10k dim random vectors

```
A  =  (-1 +1 -1 +1 +1 +1 -1 ... +1 +1 -1)
B  =  (+1 -1 +1 +1 +1 -1 +1 ... -1 -1 +1)
C  =  (+1 -1 +1 +1 -1 -1 +1 ... +1 -1 -1)
   ...
Z  =  (-1 -1 -1 -1 +1 +1 +1 ... -1 +1 -1)
#  =  (+1 +1 +1 +1 -1 -1 +1 ... +1 +1 -1)
```

Step 2) Encode text as trigrams

```
# stands for the space
```

- Use permutation to have a sequence (rotate by right by 1): THE= rrT * rH * E
- “The quick brown fox jumped over…”

```
                    Rotation of coordinates
            .->------------->------------->.
           /                                \
          /                                  \
T =  (+1 -1 -1 +1 -1 -1 . . . +1 +1 -1 -1) .   .
   H =  (+1 -1 +1 +1 +1 +1 . . . +1 -1 +1 -1) .
      E =  (+1 +1 +1 -1 -1 +1 . . . +1 -1 +1 +1)
-------------------------------------------------
THE   =    (+1 +1 -1 +1 . . . . . . +1 +1 -1 -1)
```

```
Eng += THE + HE# + E#Q + #QU + QUI + UIC + ...
```

Example - Compare and Cluster

Step 3) Measure similaritv: Cosine

Example - Computing the Profiles

Results:

- 97.3% accuracy
- English profile: after ‘th’ the closest letter is “e”, then *space*, “a”, “i”, “r” and “o”
  - Q = rrT * rH
  - X = Q * Eng
  - Closest letter vectors: X ≈ E, #, A, I, R, O

Example - Computing the Profiles

Summary:

- Start with **random** 10.000 dimensional bipolar vectors for each letter
- Compute 10.000 dimensional vectors for trigrams with **permute** and **multiply**
- **Add** all trigram vectors together to form a profile vector for languages and test sentences
- Compare using cosine

Similarities to DL

Similarity:

- Statistical learning from data
- Data can be noisy
- Both use high-dimensional vectors (NNs bog down when dim. Gets 10k?)

Differences:

- Relies on mathematical theory, not on heuristics
- New codewords are made from existing ones in a single application of HDV operations
- Memory is a separate function (stores and retrieves data as nn search)

Therefore:

- Transparent, Incremental, Scalable

Forecast

- New computers will be built
- Simpler, faster ML algorithms
- Vision and Language
- Multi-sensor integration & Robots

Forecast

- New computers will be built
- Simpler, faster ML algorithms
- Vision and Language
- Multi-sensor integration & Robots

Note! Different arithmetic is possible

VSAs	Ref.	Symbol Set	Multiplication	Addition	Permutation	Similarity metric
BSC	[6]	dense binary vectors	elementwise XOR	majority function	rotation	Hamming
MAP	[7]	dense bipolar vectors	elementwise multiplication	elementwise addition	rotation	cosine
HRR	[12]	unit vectors	circular convolution	elementwise addition	none	dot product
FHRR	[5]	complex unitary vectors	elementwise multiplication	angle of sum	circular convolution	cosine
MBAT	[9]	dense bipolar vectors	vector-matrix multiplication	elementwise addition	multiple binding	dot product
BSDC	[8]	sparse binary vectors	context-dependent thinning	elementwise disjunction	rotation	overlap of vectors
GAHRR	[10]	unit vectors	geometric product	elementwise addition	none	unitary-space scalar product

Questions

Why Holographic?

- Nature of the holograms is that it distributes the information
- You can take any HDV in half, it’s going to contain the same information, but it’s going to be less precise

How is it not unsupervised learning:

- No parameter optimisation

Why 10k?

- 10k allows you to build a system that can handle a network of 1b nodes.

What are the practical applications (besides recognizing languages)

- Identifying EEG signals
- When you go to high dimensional space, you can resolve things with linear classifiers.